Trabajo Práctico 4
Grupo 01
Base de Datos

Criptografía y Enmascaramiento

- **Consigna:** Determine qué tecnología aplicar (**TDE, DDM o Hashing**) para los siguientes activos: 1) Archivos de respaldo físico, 2) Visualización de tarjetas de crédito para soporte técnico, y 3) Almacenamiento de contraseñas.
- **Objetivo:** Diferenciar entre la protección contra robo físico y la limitación de visibilidad en la capa de aplicación.

1) Archivos de respaldo físico:

Tecnología a aplicar: TDE (Transparent Data Encryption)

Por qué: TDE está diseñado específicamente para proteger datos en reposo (*data at rest*). Cifra los archivos de la base de datos (`.mdf`, `.ldf`) y, fundamentalmente, los archivos de respaldo (backups) de forma transparente para la aplicación. Si un atacante roba físicamente el disco rígido, la cinta de respaldo o se infiltra en el servidor y se lleva el archivo de backup, no podrá restaurarlo ni leer su contenido en otro servidor sin las claves de cifrado maestras.

Cabe destacar que el cifrado de datos ocurre a nivel de base de datos, no de tablas y cifrará todos los datos que se escriben en el disco duro: tablas, índices, vistas y hasta los archivos de transacciones (`.ldf`) y copias de seguridad.

Escenario A: Se cifra la base de datos DESPUÉS de crear las tablas

Es el caso más habitual. Cuando se decide que la base de datos está lista, se ejecuta el comando `SET ENCRYPTION ON`.

El motor empieza a escanear los archivos de datos en segundo plano y va cifrando todo lo que ya existía, página por página.

- **Ventaja:** No altera tu flujo de desarrollo inicial.
- **Desventaja:** Si la base de datos ya tiene terabytes de información real, ese proceso inicial de cifrado consumirá recursos de disco (I/O) durante un rato hasta que termine el escaneo completo.

Escenario B: Se cifra la base de datos ANTES de crear las tablas (DB vacía)

Se crea la base de datos en blanco, se configura TDE y luego CREATE TABLE.

- Como la base de datos está vacía, el proceso inicial tarda un milisegundo. Luego, cualquier tabla creada o dato insertado se cifrará en instante en que se guarde en el disco.
- **Ventaja:** Es ultra rápido de activar porque no hay datos viejos que procesar.
- **Desventaja:** Ninguna, pero se debe tener listo los certificados y las claves en el entorno de desarrollo o servidores de prueba desde el principio.

Postgre, a diferencia de otras bases de datos como SQL Server, no cuenta con TDE nativo. Una opción es usar la extensión `pg_tde` y configurar previamente una Key Provider en el servidor, luego para cifrar (a nivel de tabla):

```
CREATE EXTENSION pg_tde;
CREATE TABLE productos (
        id INT,
        codigo_secreto TEXT )
USING pg_tde;
```

2) Visualización de Tarjetas de Crédito:

Tecnología a aplicar: DDM (Dynamic Data Masking / Enmascaramiento Dinámico de Datos)

Por qué: Aquí el problema no es el robo físico del servidor, sino el principio de menor privilegio en la capa de aplicación. Un usuario de soporte técnico necesita validar la identidad de un cliente o rastrear un problema, pero no necesita ver los 16 dígitos de su tarjeta de crédito. DDM ofusca el dato al vuelo (por ejemplo, mostrando XXXX-XXXX-XXXX-4321) según el rol del usuario que realiza la consulta, mientras que el dato real en el disco sigue estando completo y utilizable para procesar pagos.

Para visualizar un ejemplo en Postgre, podemos utilizar la extensión “anon” que permite aplicar reglas de enmascaramiento en las tablas usando etiquetas de seguridad.
Tomando en cuenta que se tiene creada de antemano una tabla clientes con todos los datos de una persona, incluido el número de su tarjeta de crédito, podemos hacer:

```
SECURITY LABEL FOR anon ON COLUMN clientes.tarjeta_credito
        IS 'MASKED WITH FUNCTION anon.partial(tarjeta_credito, 0, "**********", 4)';
```

Con esto enmascaramos la columna tarjeta_credito con el caracter “*”, exceptuando los últimos cuatro dígitos.
Otra ventaja es que podremos crear diferentes etiquetas para distintos roles, alternando entre “MASKED” o “UNMASKED”

Una ventaja crítica de este enfoque es que el enmascaramiento se procesa en la memoria RAM del servidor de base de datos justo antes de enviar los resultados, lo que reduce drásticamente el tráfico de red al no transferir strings pesados en texto plano. Además, al implementar `anon`, la estructura de la tabla original permanece idéntica y los índices existentes sobre la columna `tarjeta_credito` siguen siendo utilizables por el optimizador de consultas. Esto asegura que la aplicación conserve su velocidad de respuesta habitual sin necesidad de reescribir la lógica de los componentes de software actuales

3) Almacenamiento de Contraseñas:

Tecnología a aplicar: Hashing

Por qué: Las contraseñas nunca deben guardarse en texto plano ni cifrarse con métodos reversibles (como TDE). Se utiliza Hashing (idealmente combinado con *Salt* para evitar ataques de diccionario) porque es un algoritmo de una sola vía. La base de datos no necesita saber cuál es tu contraseña; solo necesita guardar el resultado del hash. Cuando inicias sesión, el sistema aplica el mismo algoritmo a lo que escribiste y compara los hashes. Si el servidor se ve comprometido, el atacante solo obtiene cadenas de texto irreversibles.

Para este caso Postgre nos provee una extensión llamada “pgcrypto ”, que se puede activar así:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Para la tabla de ejemplo tenemos, donde es importante notar que el tipo de la columna a hashear concuerde con el texto resultante de retorno(dependera del algoritmo), aunque también podemos no especificarlo usando tipo TEXT:

```
CREATE TABLE usuarios (
        id SERIAL PRIMARY KEY,
        username VARCHAR(50) UNIQUE NOT NULL,
        password_hash TEXT NOT NULL );
```

Y finalmente la inserción de elementos quedaria de esta forma:

```
INSERT INTO usuarios (username, password_hash)
VALUES (
        'jondoe', crypt('contraseñasupersecreta', gen_salt('bf', 10))
);
```

Donde el primer parámetro de gen_salt sera el algoritmo escogido para cifrar(bf es blowfish), y el segundo parámetro, el factor de costo(10 es el estándar).